

AI Audit Report — 5-section Template per Artifact

Mandatory appendix for every AI-assisted homework (HW#01–HW#06, and Seminar).

Adapted from Med Kharbach, PhD (2026) — AI Use Policy Templates for Higher Education. CC BY-NC-SA 4.0.
This adaptation is prepared for FIT@HCMUS – CS423 / CSC15003 Software Testing course.

1. Student Information

Field	Value
Student name (printed):	Trương Lý Khải
Student ID:	23127061
Class / Cohort:	23KTPM3
Assignment ID (e.g., HW#00, HW#02):	HW02-AI
Assignment date:	2/6/2026
AI tool(s) used:	Github Copilot
AI tool(s) used:	☒ Yes [] No

2. Instructions

- Add one row per AI-generated artifact (test case, script, checklist, OpenAPI spec, JMeter plan, etc.).
- Paste the verbatim prompt — DO NOT paraphrase.
- Paste the verbatim AI output (or include a labelled screenshot in the report).
- Tag the verdict: VALID / INVALID / INCOMPLETE.
- Reasoning must cite a course slide, ISTQB section, or technical RFC.
- Show the corrected artifact with the change highlighted.
- Sample rows are in italic — replace them before submission.

3. Audit Table (Tool: Github Copilot)

(1) Prompt	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
I want to create an agent skill for domain testing, this skill will be used to apply the methods of domain testing and equivalence partitioning	I'm creating the requested skill from the workflow you described and will also add the required session log entry in the project logs. The workspace does not already contain a skill definition, so I'm checking the repository's conventions before placing the new one. I've drafted the skill structure and am saving it in the workspace so it can be	VALID	The rules are already predefined and they follow the instructions given from the course slide "Domain Testing"	None

(1) Prompt	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
whenever appropriate for making tests, this skill will also record and write down any session of test making to logs.md following the rules of logs editing. This agent skill will be purely black-box, meaning it cannot access nor have a clear grasp on the source code, however, it can gain access to the program's info and setup, as well as the requirements. The process will be as follows: - The agent will be given (or is already given) info about the program and how it is run - The agent will be assigned a functional requirement to create test-cases.	used directly for future test-design sessions. The skill file is in place, and I'm appending the required session log entry to the project log now. Edited logs.md+17-1 ✅ Skill created and logged The new skill is now available at .github/skills/domain-testing/SKILL.md, and the required session entry was added to setup/logs/logs.md. What the skill covers - Black-box domain testing - Equivalence partitioning - Boundary value analysis - Test artifact creation from the provided templates - Traceability matrix updates - A step-by-step reasoning note - Session logging in the project log format - Next step You can invoke it in future sessions by referencing the skill name: domain-testing.			

(1) Prompt	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	- Following the methods of black-box testing, domain testing and equivalence partitioning with boundary values, a list of equivalence classes will be identified and similarly, a list of test cases from those classes. - Using the templates, the agent will first create tables (if not yet existed) and add tests and equivalence classes, while updating the 3 types of traceability matrices about functional requirements, equivalence classes and test cases. - The agent will then write down in a file in step-by-step details about how techniques of domain testing and boundary value analysis			

(1) Prompt	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	are used for test cases creation. This part must be written with clear, precise concepts, maintaining truths and avoiding hallucinations. - If possible, the agent can use scripts to view and execute some test cases, otherwise this will be done manually by the user. - The bug screenshots and documentation will be done primarily by the user. - After any session of the agent, a log must be recorded within logs.md following the format. Note that after this session also, a log should also be written in logs.md			
can i modify it to	I'm adjusting the skill so it explicitly points at the relevant folders and makes	VALID	This command is made so that it	None

(1) Prompt	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
automatically access the info folder, the skills folder and the templates folder for instructions, while also automatically access the logs folder and the tables folder to write down the outputs?	the workflow more automatic for reading and writing. I'm making the workflow explicit so the skill will look in the required folders by default and write its outputs to the designated locations. Yes — that can be made explicit in the skill. I updated the skill so it now: reads from the info, skills, and templates folders by default writes outputs to the logs folder and a dedicated tables folder uses those locations as the expected workflow for test artifacts and session logging The relevant change is in .github/skills/domain-testing/SKILL.md, and I also created setup/tables for generated table outputs.		can access repository information. Testing rules does not apply here.	
i've added the tables file while also added this file, the agent should have access to this file to write the explanation of explaining how the equivalence classes and boundary values are used	I'm updating the workflow so it explicitly includes the new explanation file and then I'll record this session in the project log. I'm making the skill explicitly aware of the explanation file location so future runs will use it as part of the workflow. I'm appending this latest session to the log so the workflow update is documented as requested. Edited logs.md+7-0 The latest session has been added to setup/logs/logs.md, and the skill now points to report-deliverables/domain_bva_raw.txt for the explanation output.	INCOMPLETE	While the agent knows the basic guideline of the course slides Domain Testing and Equivalence Partitioning, its analyses are still flawed and require review.	Once all raw versions are made, I compiled the files and worked out the errors, added context and restructured the file to better fit the guidelines.

(1) Prompt	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
run the domain-testing skill for FR-05	I'm applying the skill to FR-05 by gathering the relevant requirements and then drafting the equivalence classes, test cases, and supporting notes in the project artifacts.			
	I'm collecting the required artifacts and then I'll write the FR-05 test design into the project's test tables and reasoning note.			Modified many equivalent classes and test cases to reflect the right usage of Domain
	I've gathered the FR-05 requirements and I'm writing the derived equivalence classes, test cases, matrix links, and the explanatory note into the deliverables folder.		Many instances of misunderstanding of Domain	Testing and Boundary Value Analysis. Full details can be found within the Domain
	The FR-05 artifacts are now drafted; I'm updating the session log with this run so the workflow is fully recorded.	INCOMPLETE	Testing and Boundary Values found within course slide Domain Testing.	Testing and Boundary Value Analysis. Full details can be found within the Domain
	FR-05 has been processed with the domain-testing workflow.			Testing and Boundary Value Analysis report.
	Outputs created Equivalence classes: setup/tables/equivalence_class_table.md Test cases: setup/tables/test_table.md Requirement traceability: setup/tables/test_requirement_matrix.md Equivalence-class traceability: setup/tables/test_class_matrix.md Explanation note: report-deliverables/domain_bva_raw.txt Session log The run was also appended to setup/logs/logs.md.			
run the domain-testing skill on FR-14	I'm gathering the relevant requirements and templates for FR-14 so I can produce the requested domain-testing artifacts and update the logs. The relevant API surface for admin category and product management is	INCOMPLETE	Many instances of misunderstanding of Domain Testing and Boundary Values found within	Modified many equivalent classes and test cases to reflect the right

(1) Prompt	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	clear, and I'm now translating that into equivalence classes, boundary-aware tests, and the requested table updates. I've narrowed this to the category-management rule set and I'm aligning the new cases with the repository's existing table format before writing them into the project artifacts. The requirement is specific enough to derive a compact set of classes, and I'm adding the new cases in the same artifact style as the earlier FR-05 work so the matrices stay consistent. I'm writing the FR-14 equivalence classes and test cases into the project tables now, along with the supporting reasoning note and the session log entry. The main artifact updates are in place; I'm resolving the remaining matrix and log formatting so the append operation is fully consistent with the repository's existing structure. The remaining step is a small formatting pass to add the FR-14 coverage into the class traceability matrix and append the new log entry at the end of the session log. The FR-14 artifacts have been appended and I'm doing a quick verification pass over the updated tables and notes before I hand back the result.		course slide Domain Testing.	usage of Domain Testing and Boundary Value Analysis. Full details can be found within the Domain Testing and Boundary Value Analysis report.

4. Summary of AI Accuracy

Aggregate the verdicts from Section 3 and complete the table below.

Metric	Count	Percentage
Total AI-generated artifacts audited	5	100%

Metric	Count	Percentage
VALID (correct, accepted as-is)	2	40%
INVALID (wrong; rejected)	0	0%
INCOMPLETE (acceptable after edits)	3	60%

5. Conclusion — When should AI be used (or not)?

In general, there exist potential for a well-crafted agent to automate and handle streamlined and complicated tasks. However, creating such an agent is difficult and the average agent most of the time still generates too much errors and defects. The agent skill is useful at the beginning to jumpstart some test case idea with grounded reasoning. Depending on the simplicity of the requirements, the agent may even get the job done first try. The more likely situation, however, is that the user must still manually review each test case manually, while having a general grasp of the relevant knowledge to adjust the batch of tests in the right direction. I can see myself reusing the agent in the future, but due to the usefulness of the agent being limited and the limit of tokens, I do not think it will be used often.

6. Mandatory Disclosure (paste verbatim)

"Test cases for FR-05 and FR-14 was initially generated by GitHub Copilot; I reviewed and modified all of those test cases, added edge cases TC05, TC06, TC19. Test cases for FR-11, FR-15, FR-21 and all reports were written entirely by me. The detailed AI Audit Report is attached as Appendix A. I confirm I did not use AI to generate any artifact listed in the prohibited category."

Signature

Field	Value
Student name (printed):	Trương Lý Khải
Student ID:	23127061
Class / Cohort:	23KTPM3
Course:	CS423 / CSC13003 – Software Testing
Instructor:	Mrs. Trần Thị Bích Hạnh
Date:	29/06/2026

Signature:



References

- Kharbach, M. (2026). AI Use Policy Templates for Higher Education. CC BY-NC-SA 4.0.
- ISTQB Foundation Level Syllabus (latest version).
- Hardman, P. (2025). A Post-AI Learning Taxonomy.
- Fuster Rabella, M. (2025). OECD Education Working Paper No. 338.

- Perkins, M., Roe, J., & Furze, L. (2025). AI Assessment Scale.
- Anthropic (2025). Building reliable AI test agents — engineering blog.
- DeepEval & Promptfoo documentation — testing frameworks for LLM systems.